

Contrato de Endpoints — Módulos Clientes e Inmuebles

Versión: 1.0
Fecha: 2026-04-28
Estado: Borrador para revisión del equipo
Frontend: Emily Perea Córdoba
Backend: Por definir

Convenciones generales

Idénticas al contrato de Contratos y Pagos/Mora.

Parámetro	Valor
Base URL	NEXT_PUBLIC_API_URL (variable de entorno)
Formato de fecha	ISO 8601: YYYY-MM-DD
Moneda	Pesos colombianos (COP), siempre como number entero
Paginación	page (base 1) y limit en query params; respuesta incluye total y totalPaginas
Errores	Objeto { mensaje: string, codigo?: string } en el campo error
Autenticación	Bearer token en header Authorization (JWT — interceptor pendiente en src/proxy.ts)
Content-Type	application/json salvo endpoints de upload de archivos (multipart/form-data)

Estructura de respuesta estándar

```
// Éxito – recurso único
{ data: T }

// Éxito – listado paginado
{
  data: T[]
  total: number // total de registros que coinciden con el filtro
  pagina: number
  totalPaginas: number
}

// Error
{
  error: {
    mensaje: string // mensaje legible
    codigo?: string // código interno opcional, ej: "CLIENTE_NO_ENCONTRADO"
```

```
}  
}
```

Tipos compartidos

```
type TipoPersona = "natural" | "juridica"  
  
type TipoCliente = "propietario" | "arrendatario" | "prospecto" | "codeudor"  
  
type TipoDocumento = "CC" | "NIT" | "CE" | "PAS"  
  
type TipoInteraccion = "visita" | "llamada" | "mensaje" | "nota"  
  
type TipoInmueble = "casa" | "apartamento" | "local" | "otro"  
  
type ModalidadInmueble = "arriendo" | "venta" | "ambos"  
  
type EstadoInmueble =  
  | "disponible"  
  | "arrendado"  
  | "en_proceso_venta"  
  | "vendido"  
  | "en_mantenimiento"
```

Módulo: Clientes

1. Listar clientes

```
GET /clientes
```

Usado por **UI-CL01** (tabla con filtros).

Query params (todos opcionales):

Param	Tipo	Descripción
busqueda	string	Filtra por nombre completo o documento (búsqueda parcial, case-insensitive)
tipo	TipoCliente	Filtra por uno de los tipos del cliente (<code>propietario</code> , <code>arrendatario</code> , <code>prospecto</code> , <code>codeudor</code>)
tipoPersona	TipoPersona	Filtra por <code>"natural"</code> o <code>"juridica"</code>
activo	boolean	Si se omite, devuelve todos. <code>true</code> solo activos, <code>false</code> solo inactivos

Param	Tipo	Descripción
page	number	Página (default: 1)
limit	number	Resultados por página (default: 20)

Respuesta exitosa 200:

```
{
  data: ClienteResumen[]
  total: number
  pagina: number
  totalPaginas: number
}

interface ClienteResumen {
  id: string
  tipoPersona: TipoPersona
  nombre: string // nombre completo o razón social
  documento: string
  tipoDocumento: TipoDocumento
  tipos: TipoCliente[]
  telefono: string
  email: string
  ciudad: string
  fechaRegistro: string // ISO date
  representanteLegal?: string // solo persona jurídica
  activo: boolean
}
```

2. Obtener detalle de cliente

```
GET /clientes/:id
```

Usado por **UI-CL03** (tab de datos generales). Devuelve el cliente completo incluyendo resúmenes de contratos e inmuebles para las otras tabs.

Respuesta exitosa 200:

```
{
  data: ClienteDetalle
}

interface ClienteDetalle {
  id: string
  tipoPersona: TipoPersona
```

```

nombre: string
documento: string
tipoDocumento: TipoDocumento
tipos: TipoCliente[]
telefono: string
email: string
ciudad: string
fechaRegistro: string
representanteLegal?: string // solo persona jurídica
activo: boolean

// Resúmenes para las otras tabs (evitan llamadas extra al cargar la página)
totalContratos: number
totalInmuebles: number
totalInteracciones: number
}

```

Errores:

- 404 — Cliente no encontrado

3. Crear cliente

POST /clientes

Usado por **UI-CL02** (formulario de registro).

Body:

```

// Persona natural
{
  tipoPersona: "natural"
  nombre: string // nombre completo
  documento: string
  tipoDocumento: "CC" | "CE" | "PAS"
  tipos: TipoCliente[] // mínimo uno
  telefono: string
  email: string
  ciudad: string
}

// Persona jurídica
{
  tipoPersona: "juridica"
  nombre: string // razón social
  documento: string // NIT sin dígito de verificación
  tipoDocumento: "NIT"
}

```

```
tipos: TipoCliente[]
telefono: string
email: string
ciudad: string
representanteLegal: string // requerido para jurídica
}
```

Respuesta exitosa 201:

```
{
  data: ClienteResumen // cliente recién creado, con id asignado
}
```

Errores:

- 400 — Datos inválidos (campos faltantes, formato de documento incorrecto)
 - 409 — Ya existe un cliente con ese documento
-

4. Editar cliente

```
PUT /clientes/:id
```

Usado por **UI-CL02** (formulario en modo edición). El body tiene la misma estructura que el **POST /clientes**.

Body: Mismo esquema que **POST /clientes** (todos los campos reemplazados).

Respuesta exitosa 200:

```
{
  data: ClienteResumen
}
```

Errores:

- 400 — Datos inválidos
 - 404 — Cliente no encontrado
 - 409 — El documento pertenece a otro cliente
-

5. Cambiar estado (activar / desactivar)

```
PATCH /clientes/:id/estado
```

Usado desde la tabla **UI-CL01** (toggle de estado) y desde el detalle **UI-CL03**.

Body:

```
{
  activo: boolean
}
```

Respuesta exitosa 200:

```
{
  data: {
    id: string
    activo: boolean
  }
}
```

Errores:

- **404** — Cliente no encontrado
 - **400** — El cliente tiene contratos activos y no puede desactivarse
-

6. Listar contratos del cliente

```
GET /clientes/:id/contratos
```

Usado por **UI-CL03** (tab de contratos).

Respuesta exitosa 200:

```
{
  data: ContratoClienteResumen[]
}

interface ContratoClienteResumen {
  id: string
  referencia: string // ej: "CTR-2025-001"
  tipo: "arriendo" | "promesa_compraventa"
  estado: string // ver EstadoContrato en contrato-endpoints.md
  inmueble: string // nombre descriptivo
  direccion: string
  rol: TipoCliente // rol del cliente en este contrato
  (propietario, arrendatario, codeudor)
  fechaInicio: string
}
```

```
    fechaFin: string
  }
```

7. Listar inmuebles del cliente

```
GET /clientes/:id/inmuebles
```

Usado por **UI-CL03** (tab de inmuebles asociados). Devuelve los inmuebles donde el cliente figura como propietario.

Respuesta exitosa 200:

```
{
  data: InmuebleClienteResumen[]
}

interface InmuebleClienteResumen {
  id: string
  tipo: TipoInmueble
  modalidad: ModalidadInmueble
  estado: EstadoInmueble
  direccion: string
  ubicacion: string           // ej: "Bogotá – Chapinero"
  area: number                // m²
  precio: number              // COP – valor de arriendo o precio de venta
}
```

8. Listar interacciones del cliente

```
GET /clientes/:id/interacciones
```

Usado por **UI-CL03** (tab de historial) y para mostrar el historial actualizado tras registrar una interacción.

Query params (todos opcionales):

Param	Tipo	Descripción
tipo	TipoInteraccion	Filtra por tipo de interacción
desde	string	ISO date — interacciones desde esta fecha
hasta	string	ISO date — interacciones hasta esta fecha
page	number	Página (default: 1)

Param	Tipo	Descripción
limit	number	Resultados por página (default: 50)

Respuesta exitosa 200:

```
{
  data: Interaccion[]
  total: number
  pagina: number
  totalPaginas: number
}

interface Interaccion {
  id: string
  tipo: TipoInteraccion
  fecha: string // ISO date
  hora?: string // "HH:mm" – puede estar ausente en notas
  descripcion: string
  asesor: string // nombre del asesor que registró
  inmueble?: string // nombre descriptivo, solo si tipo ===
"visita"
  inmuebleId?: string // ID del inmueble, solo si tipo === "visita"
}
```

9. Registrar interacción

```
POST /clientes/:id/interacciones
```

Usado por **UI-CL04** (sheet de registrar interacción). Crea una interacción de tipo visita, llamada, mensaje o nota.

Body:

```
{
  tipo: TipoInteraccion
  fecha: string // ISO date – no puede ser fecha futura
  hora: string // "HH:mm" – requerido
  descripcion: string // requerido, texto libre
  inmuebleId?: string // requerido si tipo === "visita"
}
```

Respuesta exitosa 201:

```
{
  data: Interaccion           // interacción recién creada, con id asignado
}
```

Errores:

- 400 — Campos requeridos faltantes o fecha inválida
- 404 — Cliente no encontrado
- 404 — Inmueble no encontrado (si se pasó `inmuebleId`)

Módulo: Inmuebles

10. Listar inmuebles

```
GET /inmuebles
```

Usado por **UI-I01** (tabla de inmuebles con filtros). Esta es la versión completa del endpoint resumido en `contrato-endpoints.md` §16.

Query params (todos opcionales):

Param	Tipo	Descripción
<code>busqueda</code>	<code>string</code>	Filtra por dirección o ubicación (búsqueda parcial, case-insensitive)
<code>tipo</code>	<code>TipoInmueble</code>	Filtra por tipo (<code>casa</code> , <code>apartamento</code> , <code>local</code> , <code>otro</code>)
<code>modalidad</code>	<code>ModalidadInmueble</code>	Filtra por modalidad (<code>arriendo</code> , <code>venta</code> , <code>ambos</code>)
<code>estado</code>	<code>EstadoInmueble</code>	Filtra por estado exacto
<code>disponibles</code>	<code>boolean</code>	Si <code>true</code> , solo inmuebles en estado " <code>disponible</code> "
<code>propietarioId</code>	<code>string</code>	Filtra por ID del propietario (usado en el detalle de cliente)
<code>page</code>	<code>number</code>	Página (default: 1)
<code>limit</code>	<code>number</code>	Resultados por página (default: 20)

Respuesta exitosa 200:

```
{
  data: InmuebleResumen[]
  total: number
  pagina: number
  totalPaginas: number
  resumenEstados: Record<EstadoInmueble, number> // conteo por estado, para
```

```

las tarjetas del encabezado
}

interface InmuebleResumen {
    id: string
    tipo: TipoInmueble
    modalidad: ModalidadInmueble
    estado: EstadoInmueble
    publicado: boolean
    direccion: string
    ubicacion: string           // ej: "Bogotá – Chapinero"
    area: number                // m²
    precio: number              // COP
    propietario: string         // nombre completo o razón social
    propietarioId: string
    fechaRegistro: string       // ISO date
    fotoPrincipal?: string      // URL de la primera foto, si existe
}

```

11. Obtener detalle de inmueble

```
GET /inmuebles/:id
```

Usado por **UI-I03** (detalle con tabs: info, fotos, historial). Devuelve el objeto completo incluyendo todas las fotos y el historial de cambios.

Respuesta exitosa 200:

```

{
  data: InmuebleDetalle
}

interface InmuebleDetalle {
    id: string
    tipo: TipoInmueble
    modalidad: ModalidadInmueble
    estado: EstadoInmueble
    publicado: boolean
    direccion: string
    ubicacion: string
    area: number
    precio: number
    propietario: string
    propietarioId: string
    fechaRegistro: string
    coordenadas?: [number, number] // [lat, lng] – opcional
}

```

```

    fotos: FotoInmueble[]
    historial: CambioHistorial[]

    // Contrato activo actual, si existe
    contratoActivo?: {
      id: string
      referencia: string
      tipo: "arriendo" | "promesa_compraventa"
      estado: string
      fechaFin: string
    }
  }

  interface FotoInmueble {
    id: string
    url: string
    descripcion?: string
  }

  interface CambioHistorial {
    id: string
    fecha: string // ISO date
    campo: string // nombre del campo modificado, ej: "estado",
    "precio"
    valorAnterior: string
    valorNuevo: string
    usuario: string // nombre del asesor
  }

```

Errores:

- **404** — Inmueble no encontrado

12. Registrar inmueble

POST /inmuebles

Usado por **UI-102** (formulario de registro).

Body:

```

{
  tipo: TipoInmueble
  modalidad: ModalidadInmueble
  direccion: string
  ubicacion: string // ej: "Bogotá – Chapinero"
  area: number // m², entero positivo
}

```

```
precio: number // COP
propietarioId: string // ID del cliente propietario
publicado: boolean // si el inmueble es visible en el portal
coordenadas?: [number, number]
}
```

Respuesta exitosa 201:

```
{
  data: InmuebleResumen // inmueble recién creado, con id asignado y
  estado inicial "disponible"
}
```

Errores:

- 400 — Datos inválidos (área o precio negativos, campos faltantes)
- 404 — Propietario no encontrado

13. Editar inmueble

```
PUT /inmuebles/:id
```

Usado por **UI-104** (formulario en modo edición). El body tiene la misma estructura que el **POST /inmuebles**.

Body: Mismo esquema que **POST /inmuebles** (todos los campos reemplazados).

Respuesta exitosa 200:

```
{
  data: InmuebleResumen
}
```

Errores:

- 400 — Datos inválidos
- 404 — Inmueble no encontrado
- 404 — Propietario no encontrado

14. Cambiar estado del inmueble

```
PATCH /inmuebles/:id/estado
```

Permite que el asesor actualice el estado manualmente (ej. marcarlo como "en_mantenimiento" o "disponible"). Algunos estados los gestiona el backend automáticamente al crear/terminar contratos.

Body:

```
{
  estado: EstadoInmueble
  nota?: string           // motivo del cambio (libre, se registra en el
historial)
}
```

Respuesta exitosa 200:

```
{
  data: {
    id: string
    estado: EstadoInmueble
  }
}
```

Errores:

- 404 — Inmueble no encontrado
- 400 — Transición de estado no permitida (ej. pasar de "vendido" a "disponible" sin lógica de negocio)

15. Listar fotos del inmueble

```
GET /inmuebles/:id/fotos
```

Usado por **UI-I03** (tab de fotos).

Respuesta exitosa 200:

```
{
  data: FotoInmueble[]
}
```

16. Subir foto

```
POST /inmuebles/:id/fotos
Content-Type: multipart/form-data
```

Usado por **UI-I02** y **UI-I04** (galería de fotos en el formulario).

Form fields:

Campo	Tipo	Descripción
archivo	File	Imagen (JPEG, PNG, WebP — máx. 5 MB)
descripcion	string	Descripción opcional de la foto

Respuesta exitosa 201:

```
{
  data: FotoInmueble
}
```

Errores:

- 400 — Formato de archivo no soportado o tamaño excedido
- 404 — Inmueble no encontrado
- 422 — El inmueble ya tiene 20 fotos (límite máximo)

17. Eliminar foto

```
DELETE /inmuebles/:id/fotos/:fotoId
```

Usado por **UI-I02** y **UI-I04** (botón eliminar en galería).

Respuesta exitosa 204: Sin cuerpo.

Errores:

- 404 — Foto no encontrada

18. Historial de cambios del inmueble

```
GET /inmuebles/:id/historial
```

Usado por **UI-I03** (tab de historial). Ya incluido en `GET /inmuebles/:id`, pero este endpoint independiente permite paginar si el historial es extenso.

Query params (todos opcionales):

Param	Tipo	Descripción
page	number	Página (default: 1)
limit	number	Resultados por página (default: 50)

Respuesta exitosa 200:

```
{
  data: CambioHistorial[]
  total: number
  pagina: number
  totalPaginas: number
}
```

Resumen de endpoints

#	Método	Ruta	Descripción	UI
1	GET	/clientes	Listar clientes con filtros	UI-CL01
2	GET	/clientes/:id	Detalle del cliente	UI-CL03
3	POST	/clientes	Registrar cliente	UI-CL02
4	PUT	/clientes/:id	Editar cliente	UI-CL02
5	PATCH	/clientes/:id/estado	Activar / desactivar cliente	UI-CL01, UI-CL03
6	GET	/clientes/:id/contratos	Contratos del cliente	UI-CL03
7	GET	/clientes/:id/inmuebles	Inmuebles del cliente	UI-CL03
8	GET	/clientes/:id/interacciones	Historial de interacciones	UI-CL03
9	POST	/clientes/:id/interacciones	Registrar interacción	UI-CL04
10	GET	/inmuebles	Listar inmuebles con filtros	UI-I01
11	GET	/inmuebles/:id	Detalle del inmueble	UI-I03
12	POST	/inmuebles	Registrar inmueble	UI-I02
13	PUT	/inmuebles/:id	Editar inmueble	UI-I04
14	PATCH	/inmuebles/:id/estado	Cambiar estado del inmueble	UI-I03
15	GET	/inmuebles/:id/fotos	Listar fotos	UI-I03
16	POST	/inmuebles/:id/fotos	Subir foto	UI-I02, UI-I04
17	DELETE	/inmuebles/:id/fotos/:fotoId	Eliminar foto	UI-I02, UI-I04

#	Método	Ruta	Descripción	UI
18	GET	/inmuebles/:id/historial	Historial de cambios (paginado)	UI-I03

Notas de negocio para el backend

- tipos del cliente es un array:** Un mismo cliente puede ser **propietario** y **arrendatario** simultáneamente (ej. tiene un inmueble arrendado y arrienda otro). El backend debe soportar múltiples tipos y el filtro por **tipo** en **GET /clientes** debe hacer **ARRAY_CONTAINS**, no igualdad exacta.
- Estado del inmueble gestionado automáticamente:** El backend debe actualizar el estado del inmueble sin intervención del asesor en estos casos:
 - Contrato de arriendo activado → **"arrendado"**
 - Contrato de arriendo finalizado → **"disponible"**
 - Promesa de compraventa activada → **"en_proceso_venta"**
 - Escrituración completada → **"vendido"**
- precio del inmueble:** Según **modalidad**, el campo **precio** representa:
 - "arriendo"** → valor del canon mensual (COP)
 - "venta"** → precio de venta (COP)
 - "ambos"** → el backend decide cuál exponer; el frontend lo formatea según el contexto
- Historial de cambios:** El backend debe registrar automáticamente una entrada en **CambioHistorial** cada vez que se modifique un campo del inmueble (precio, estado, modalidad, etc.) o se registre un evento relevante (contrato firmado, foto subida).
- Interacciones con inmuebleId:** Al registrar una visita, el campo **inmueble** en la respuesta debe ser el **direccion** del inmueble (texto descriptivo), no solo el ID. El frontend lo muestra directamente en el historial.
- Unicidad de documento de cliente:** El par (**documento**, **tipoDocumento**) debe ser único. Dos personas no pueden compartir el mismo número de CC, pero sí pueden tener el mismo número con distinto tipo (improbable en la práctica, pero el modelo lo permite).
- fotoPrincipal en listado:** Para evitar N+1 queries, el endpoint de listado **GET /inmuebles** debe incluir la URL de la primera foto directamente en el objeto **InmuebleResumen**. Si no hay fotos, devuelve **null**.
- Registrar interacción — fecha no futura:** El backend debe rechazar interacciones con **fecha** posterior a hoy. Las interacciones son registros históricos, no agendaciones futuras. (Las citas futuras son un módulo pendiente.)